

6 小时 Vibe | Agent Skill 搭建自我展示网站

🌟 作品链接

<https://renyuanyuan-design.github.io/portfolio-site/>

本文阐述我的求职网站是怎么做出来的，以及我的思考和感悟。

本文基于我与 Claude Code 的实际协作过程，由 Claude Code 整理撰写。

先看效果：



左边是把PRD喂给AI后，AI一键直出，存在问题如下：

- 中文渲染乱码
- 图片无法识别
- 简历内容提取不清
- 排版不美观
- 交互动线乱套等等

右边是经过 Skill 处理后的效果，文字渲染清晰优雅，能够完成流畅交互效果，下面分享一下探索过程。

一、痛点：我想让所有人更好的了解我

需求一开始也很模糊：

"我是个准备求职的应届生，有多段实习，还有一堆校园经历和证书。我想做一个网站，让所有人点击链接就能看到我的经历，更好的了解我"

翻译成工程语言，这个模糊需求里其实藏着几个硬约束：

- 必须是个**静态网站**（零后端、零构建，GitHub Pages 一键托管）
- 必须在**国内网络**可访问（GitHub Pages 国内其实能开，但慢；所以还要考虑离线包和 Vercel 备份）
- 必须**图文混排、视觉过关**（求职作品集本质是"设计作品"，不是纯文字简历）
- 必须**手机上也好看**（因为你不知道网站到底是在电脑上还是在手机上被打开）

最大的卡点就是：**我不会写前端代码，也没法找人协作开发。**

因此只能求助AI。

二、常规路线走不通

最开始我尝试了几个方向：

- **用现成建站工具（Wix / 小红书主页 / Canva 简历页）**：拖拖拽拽能做，但要么模板味太重、不像"我的作品"，要么导出后还是一张张图，没法做一些比较有特色交互，而且域名不干净、国内访问慢。
- **让 AI 一次性"生成一个完美网站"**：它能吐出一个看起来不错的单页，但细节全是坑：图片比例乱裁、移动端挤成一团、字体加载失败变方块、链接部署上去 404。**看起来它擅长"画个还不错的大饼"，而不是"交付一个直接可以使用的产品"。**

到这里，其实问题的本质是："做一个能直接点击查看的求职自我介绍网站"是一个模糊的大需求，**直接一次性解决很难**。它混杂了设计、前端、部署、访问、适配五件不同的事。

三、灵感：把"网站"当成乐高，而不是雕塑

地图产品时有一个核心概念：**底图 + 图层**。

底图是静态的瓦片——道路、建筑、地形；POI 点、导航路线、用户标记，都是作为独立图层叠加上去的。底图不关心上面放什么 marker，marker 也不关心底图长什么样，各司其职，通过坐标层叠。



某天对着AI生成的又一个"文字乱、图片乱、视觉效果差"的页面发呆时，我突然想到：
能不能把这个网站拆成**"骨架底层"**和**"内容图层"**，由内向外，像积木一样不断叠加模块和组件，直到完整的搭好。

- 🌟 具体来说：
1. 先有一份**干净的骨架**——单页 HTML 结构 + 一套集中管理的 CSS 设计变量（配色、字体、阴影），这是**"底图"**
 2. 每一个经历、每一组文字卡、每一个弹窗模板，都是**独立的、可替换的内容模块**——这是**"图层"**
- 网站的内容是会变的（可能要增减文字或照片，或者修改交互效果），但**骨架是稳定的**。
把**"会变的"**和**"不变的"**分开，这不就是工程积木思维吗？

四、验证想法：让 AI 助手跑通流程

有了思路，下一步我开始验证可行性。

Step 1: 搭骨架——底图

我描述了需求，AI 助手生成了index.html单页结构 +style.v2.css。关键点不是页面排版，而是一开始就把**设计变量集中管理**：

代码块

```

1  :root{
2    --ink:#1a2233;
3    --blue-deep:#2b5cb8;
4    --mint:#7fd8c4;
5    /* ...所有配色 / 间距 / 圆角都在这里 */
6  }

```

这样如果后续我想换主题色，改一行就行，不用翻全文。

骨架定了，后面不断填充内容即可。

Step 2: 填内容图层——模块化

骨架稳了，接下来开始填内容，先把所有部分和模块拆分，每个部分由独立的模块构成，全部单独设计。

- 举例——比如实习经历部分的悬浮照片墙：



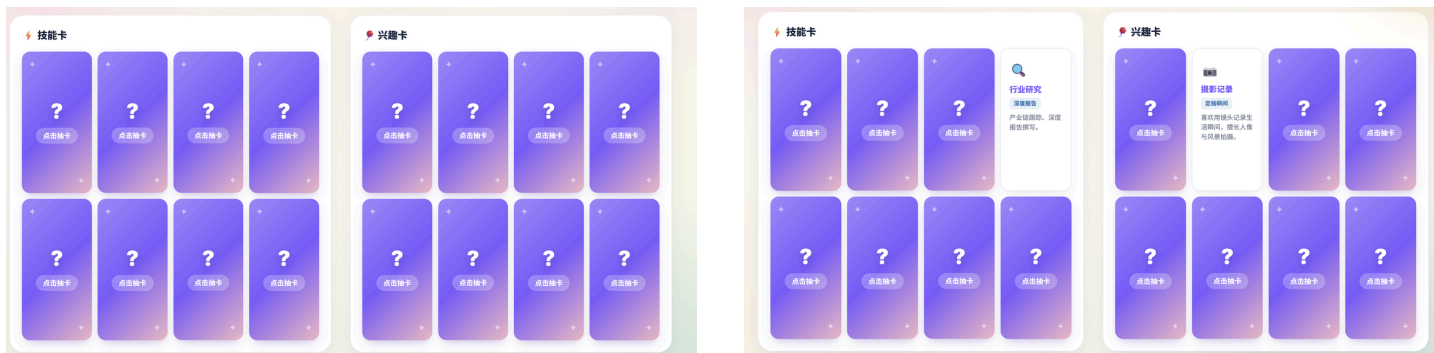
代码块

```

1  <div class="polaroids polaroids--3col" data-group="baidu">
2    <figure class="polaroid"></figure>
3    <figure class="polaroid"></figure>
4
5  </div>

```

- 举例——比如串联实习经历的纵轴
- 举例——比如技能卡牌部分的抽卡设计



Step 3: 设计细节——让用户"眼前一亮"的交互

骨架和内容都有了，但一个作品集不应该只是"能看"，而是得让人愿意多停留几秒。

我和 AI 在这个环节加了两个自研交互，作为整站的记忆点。它们其实不是一次性写对的，而是通过调试过程写出来的。

1. 鼠标行程波点（跟随光标的彩色水波轨迹）



我想要的效果是：光标划过页面，像在水面上留下一串会呼吸的彩色涟漪，颜色跟着位置渐变。

第一版 AI 写的是最简单的"每帧生成一个圆圈"：

代码块

```
1 // 初版：每帧都生成，快速划过会糊成一坨
2 window.addEventListener('mousemove', e => {
3   const r = document.createElement('span');
4   r.className = 'wave';
5   r.style.left = e.clientX + 'px';
6   r.style.top = e.clientY + 'px';
```

```
7     document.body.appendChild(r);
8   });
```

问题立刻出现：`mousemove` 每秒触发几十次，鼠标快速划过时屏幕上瞬间堆出上百个圆圈，糊成一团，而且从不销毁 → 内存暴涨。

修正一：节流成"行程"。加一个时间戳，移动事件每 90ms 才放行一次，于是快速划过会自然拉出一串疏密有致的轨迹：

代码块

```
1   let last = 0;
2   const ripple = (x, y, force) => {
3     const now = performance.now();
4     if (!force && now - last < 90) return; // 节流：沿路径留下"一串"水波
5     last = now;
6     /* ...生成波点... */
7   };
```

修正二：播完自毁，零内存泄漏。给波点绑 `animationend` 监听，动画一结束就从 DOM 移除：

代码块

```
1   r.addEventListener('animationend', () => r.remove());
```

修正三：颜色随坐标渐变。用一组锚点色，按鼠标位置取最近的主色，整页波点自带虹彩：

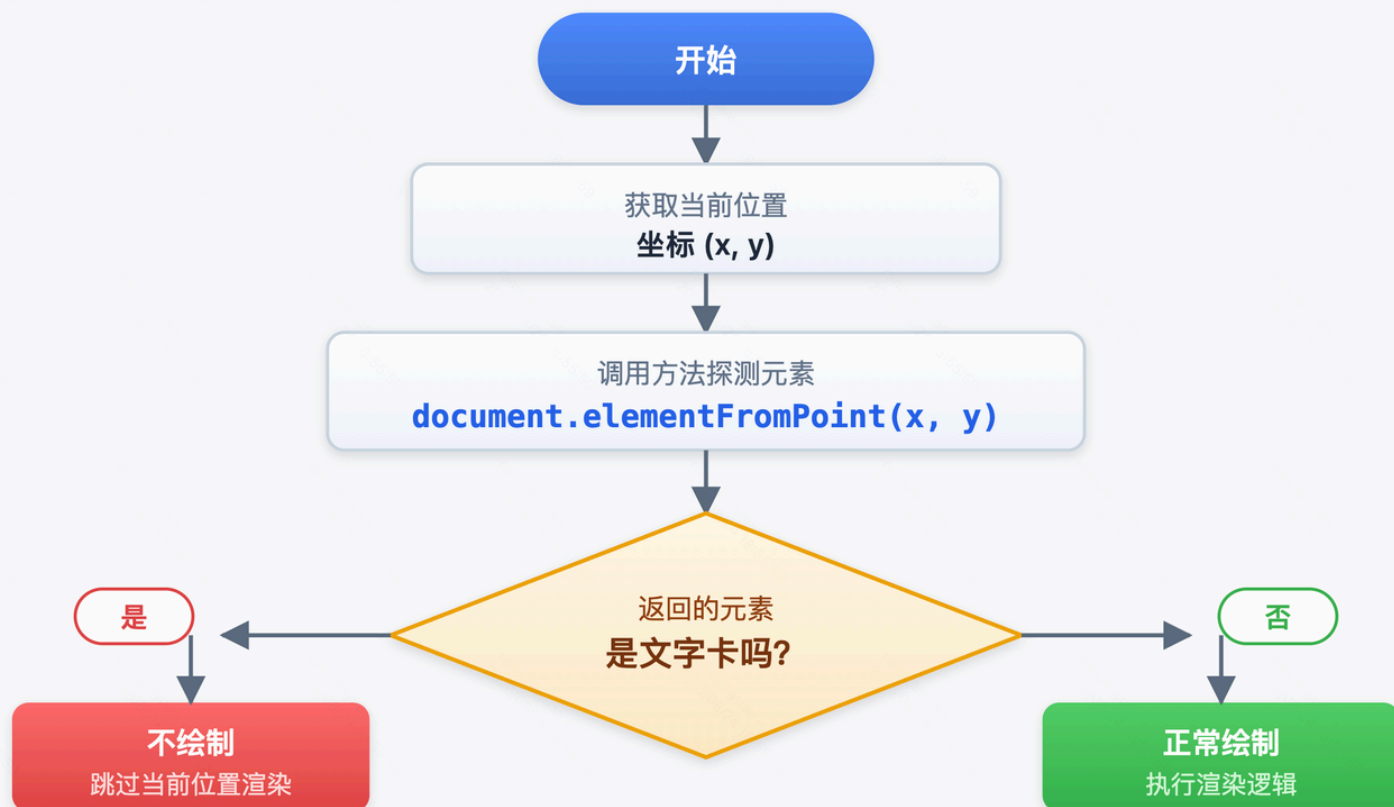
代码块

```
1   const COLORS = [
2     { c: '#fbe4ec', x: 0.12, y: 0.10 }, // 左上 粉
3     { c: '#fdf3d8', x: 0.48, y: 0.42 }, // 中   黄
4     { c: '#d9f2e8', x: 0.88, y: 0.90 }  // 右下 薄荷
5   ];
6   const colorAt = (px, py) => {
7     let best = COLORS[1], bd = Infinity;
8     COLORS.forEach(p => {
9       const d = (p.x-px)**2 + (p.y-py)**2; // 欧氏距离取最近锚点
10      if (d < bd) { bd = d; best = p; }
11    });
12    return best.c;
13  };
```


修正四：智能避让。测试时发现，波点效果如果叠加在文字卡上，会阻挡复制效果。这是因为波点用了 `mix-blend-mode:multiply` 叠加，因此，我们加了一个 `isInTextCard` 判断：

用 `document.elementFromPoint` 探测当前位置是不是文字卡，是则不绘制。

elementFromPoint 探测逻辑



核心逻辑：通过 `elementFromPoint` 命中检测，避免在文字卡区域重复绘制

代码块

```
1  const isInTextCard = (x, y) => {
2    const el = document.elementFromPoint(x, y);
3    return !(el && el.closest &&
4      (el.closest('.exp-card--text') || el.closest('.campus-card')));
5  };
```

最终配套 CSS 也很轻：

代码块

```
1  .wave{
2    position:fixed; width:0; height:0; border-radius:50%;
3    pointer-events:none; transform:translate(-50%,-50%);
4    mix-blend-mode:multiply;
5    background:radial-gradient(circle, currentColor 0%, currentColor 35%,
      transparent 70%);
```

```

6     animation:waveOut 1.1s ease-out forwards;
7   }
8   @keyframes waveOut{
9     0%{width:0;height:0;opacity:.85}
10    100%{width:380px;height:380px;opacity:0}
11  }

```

移动端用 `touchmove` / `touchstart` 同样触发，手指划过也有水波。整个交互零依赖、零图片，纯 CSS 动画 + 原生 JS。

2. 校园经历图片弹窗（数据驱动的灯箱）

校园经历部分共设置了9个组件内容，每项 1~3 张图，如果全铺在页面上，时间轴会拖得很长。

——因此，这里设计了一个**图片弹窗**，交互逻辑是：页面展示9个组件卡片 + "立即查看"按钮，点击后开启图片弹窗，再次点击可放大查看

点击图片卡【立即查看】tab

开启图片弹窗

点击具体图片

图片放大展示



关于图片弹窗中的图片展示效果，我和AI修过一个 bug：

一开始 `--pcount` 是用 `[style*="--pcount: 1"]` 这种**属性选择器**匹配的，但 JS 设置的 `style.setProperty('--pcount', '2')` 渲染出来是 `--pcount:2`（无空格），CSS 里写的是 `--pcount: 2`（有空格），两边对不上，导致图片弹窗中如果只有单图，则单图永远不居中，如下图左。



后来我们把 CSS 选择器改成同时兼容两种写法才修好，让图片居中展示，如上图右。

代码块

```
1 .campus-modal .polaroids--3col[style*="--pcount: 1"],
2 .campus-modal .polaroids--3col[style*="--pcount:1"]{ justify-content:center; }
```

为什么这两个交互值得专门写一下？

——因为它把我的**实习经历**与**产品能力**串成了完整闭环。

- 在百度实习期间，我主要负责产品原型设计，产出过乒乓大会、高考、世界杯等大量 H5 原型，参与过半屏跳转、AI 动画玩法的配置工作，积累了扎实的交互设计基础。
- 快手实习阶段，我深耕任务中心相关界面，重点跟进「快影-趣头条」三方换量场景下，圈选人群包开启弹窗对整体流量影响的对照实验。

正是这些过往实践，促使我在个人网站中落地了鼠标动效与图片弹窗两套交互逻辑。它不再是割裂的实习履历，而是把过往项目经验和个人作品打通，铺展出一条完整的能力成长线。

Step 4: 解决问题——如图片裁剪的"污染"问题

测试中发现一个典型问题：部分实习的照片想统一裁成 4:3（视觉整齐），但校园弹窗里的证书/活动照想**按原比例完整展示、不裁剪**——结果两套规则打架，造成混乱。

第一轮修法是把"不裁剪"规则直接挂在弹窗容器上：

代码块

```
1 /* ✗ 第一次尝试：以为这样就能只影响弹窗 */
2 .polaroids--3col .polaroid img{
3   aspect-ratio:auto; object-fit:contain; /* 想让弹窗里的图不裁剪 */
}
```

```
4 }
```

但实习经历的图也在 `.polaroids--3col` 里，所以这个规则把实习经历里的图片也一起改成了不裁剪——造成“图层互相污染”。

AI第一反应是加 `!important` 硬压，但更根本的问题是选择器特异性（specificity）。全局有一条早就存在的逻辑：

代码块

```
1 .polaroid img{ aspect-ratio:1.1/1; object-fit:cover; } /* 通用拍立得默认 */
```

而弹窗里那条 `.polaroids--3col .polaroid img` 虽然写了 `aspect-ratio:auto`，但只要全局那条在后面、且特异性不低，实际渲染就可能被覆盖。

排查时AI用Playwright把 `getComputedStyle(img).aspectRatio` 打出来，确认弹窗里的图算出来还是 `1.1/1` 而不是 `auto`，才定位到是特异性战争。

最终解决方案是：把作用域精确收窄 + 提高特异性：

代码块

```
1 /* 百度实习：页面版，强制 4:3 裁剪 */
2 .polaroids--3col .polaroid img{
3     aspect-ratio:4/3; object-fit:cover;
4 }
5 /* 校园弹窗：用更长的选择器 + !important 压过通用规则 */
6 .campus-modal .polaroids.polaroids--3col .polaroid img{
7     aspect-ratio:auto !important; object-fit:contain;
8 }
9 /* 单图居中：用 :has() 选择器识别"只有一张图"的情况 */
10 .campus-modal .polaroids:has(.polaroid:only-child){
11     display:flex; justify-content:center;
12 }
```

Step 5: 测试检查——在实践中改善

单端测试合格，不能覆盖全终端的体验要求。

桌面端确认网站效果没问题之后，我开始测试移动端的效果，发现两个问题：

- 照片墙挤成一团
- 弹窗被推到屏幕外

因此，与AI对话补了移动端规则：照片墙变 2 列、弹窗图片 `max-height:65vh` 防溢出、单图居中。

但这里又踩到一个**特异性陷阱**，和 Step 4 是同一类坑：

代码块

```
1  /* 移动端媒体查询里，为了让照片墙成组，早年写过一条 */
2  @media (max-width:640px){
3      .polaroid img{ aspect-ratio:1/1 !important; } /* 把所有移动端照片都压成 1:1 */
4  }
```

这条带 `!important` 的规则，把**部分实习经历在移动端的图也强制成了正方形**——桌面端是 4:3，手机端却变成 1:1。

解决方案是：写一条**特异性更高、且限定** `data-group` 的移动端豁免：

代码块

```
1  @media (max-width:640px){
2      /* 移动端百度实习：仍保持 4:3，不被上面的 1:1 规则覆盖 */
3      .polaroids--3col[data-group="baidu"] .polaroid img{
4          aspect-ratio:4/3 !important; object-fit:cover !important;
5      }
6      /* 移动端校园弹窗：原比例不裁剪 */
7      .campus-modal .polaroids.polaroids--3col .polaroid img{
8          aspect-ratio:auto !important; object-fit:contain !important; max-
          height:65vh !important;
9      }
10 }
```

这里有个值得记的经验：`!important` 是一把双刃剑。

项目初期，为了"快速让照片墙成组"的需求落地，所以AI随手加了 `!important`，结果之后改图片裁剪规则时被它反噬。正确做法是尽量靠选择器特异性解决，只在"必须覆盖第三方/旧规则"时才用 `!important`，并且**永远带上更具体的限定条件**（比如 `data-group="baidu"`），别让它无差别命中。

Step 6: 部署——让网站"能够被打开"

网站做得再好，打不开等于零。

这一步卡了最久，因为我的电脑网络环境特殊（git push 的 443 端口被封）。

查阅网上资料，结合和AI的对话调试，最终打通了一条路：

- 主入口用 **GitHub Pages**（公开仓库，提交即自动上线）
- 备用方案打 **离线 ZIP 包**，HR 双击 `index.html` 就能看，不用联网（但要注意不能依赖 Google Fonts，否则离线就白屏——我们改成了系统字体兜底）
- 备份域名预留 **Vercel**

部署的核心经验是：**交付物是"静态文件"，不是"开发环境"**。任何一台电脑、双击就能开，才是对用户友好的网站产品。

五、体验升级：把"改网站"变成"改一句话"

- 流程跑通了，但体验还不够好。每次微调位置都要改 JSON 文件，太原始了。
 - 我跟 AI 说：「能不能做一个可视化编辑器？实时预览效果。」
 - 几分钟后，一个 Web 编辑器出现了。
- 到这一步，完整的工作流已经成型：

代码块

```
1 模糊需求 → 拆成骨架+图层 → 最小单页跑通 → 实践中迭代（移动端/裁剪规则）
2  → 沉淀版本管理+离线包 → 交付友好的静态站 → 可修改编辑器
```

- 但每次都要记一堆命令和参数，还是太麻烦。
- 于是我把整个流程让 AI 自己封装成了一个 Agent Skill——`ArtText`。现在只需要说：**「把这份 PDF 做成一份个人简历网站」**
- AI 就会自动识别场景，走完整个流程，最后打开可视化编辑器让我微调。
- 这就是 Vibe Coding 的魅力：**用自然语言描述问题，让 AI 帮你构建解决方案，最后封装成可复用的能力。**
- 最后，我和 AI 共同沉淀了可复用的能力
 - **骨架与内容分离**：任何"内容会变的展示型网站"都适用：个人主页、作品集、活动主页、产品落地页。先定干净骨架 + 集中设计变量，内容当图层往里填。改内容永不动骨架。
 - **数据驱动布局**：用 `--pcount` 这类变量让 CSS 自己决定排几列，比写死 N 套布局省十倍维护成本。适用于任何"数量不固定的卡片列表"。
 - **真机验收**：每次改动用 Playwright 模拟桌面 + iPhone 双视口逐项目截图验收，而不是靠肉眼猜。这条适合所有前端改动——**人眼会骗人，截图不会。**
 - **静态交付优先**：能做成纯静态文件 + Pages 托管的，绝不引入框架和构建。用户使用自己的电脑，双击即开才是王道。
 - **版本即资产**：git tag + 版本文档，让"试错"零成本。敢大改，因为随时能回滚。

- **轻量自研交互，比套模板更有记忆点：**鼠标行程波点、数据驱动灯箱这类交互，不用任何库、几十行原生 JS 就能做。它们的价值不是"炫技"，而是**让用户在划过页面的 3 秒里记住你**。任何展示型站点都可以加一两个"零成本的惊喜"，关键是交互要服务内容（波点避让文字卡），而不是喧宾夺主。

六、启发

拆解问题

"做一个求职网站"是模糊需求，直接做很难。拆成"骨架、内容图层、移动适配、部署通道"后，每一步都可控。这和做产品需求拆解是同一个动作。

迭代式探索

发现让AI一键"直接生成网站"不可行后，马上调整思路，先跑通一个能看的单页，再逐步加弹窗、加自研交互（波点、灯箱）、加移动适配、加版本管理。裁剪规则、移动端豁免、波点避让文字卡，发现问题及时改进。

能力迁移

鼠标行程波点和图片弹窗两个交互，不是凭空想象出来的，都是基于我的实习经历沉淀出来的。学到的东西只有灵活运用，才算是真正掌握。

AI 是很好的协作者

整个过程中，我几乎没写一行代码。CSS 选择器怎么写、Playwright 脚本怎么跑、GitHub PAT 怎么配——我完全不需要懂。我只负责三件事：**定义需求、提供思路、验收结果。**

AI 助手负责所有技术实现，甚至会主动发现我没想到的边界情况，这种分工让我能聚焦在真正重要的事上：这个站**传达出的人设对不对、用户有没有兴趣**……技术细节？都交给 AI！这种配合是成功的关键。

所以问题的解法，往往不在问题本身所属的领域。

"网站怎么修改"的答案，来自地图产品的"底图+图层"；

"怎么让外行也能交付产品"的答案，来自把 AI 当协作者而不是当代码生成器。

过去攒下的每一份经验，都可能在某个完全不相干的时刻里救我一命，这就是回响。

（注：部分内容可能由 AI 生成）